DevSecOps with AWS - 888

LINUX 3

SHELL SCRIPTING 3

27-JAN-2026 Session-14 3

Inode Symlink Hardlink 3

Version control system 10

Centralised vs distributed 12

Git commands 12

28-JAN-2026 Session-15 12

ANSIBLE 19

Session-23:09-FEB-2026 26

Variables from argos 26

Variables preference 27

Data types 28

Conditions 28

Shell vs command module 29

Register output 30

Filters 30

Session-24:10-FEB-2026 31

Error handling 32

Session-29:18-FEB-2026 33

Include vs import role 33

Dynamic inventory 33

Vault 33

Parallel execution 33

Ansible disadvantages 34

Session-30:19-FEB-2026 35

TERRAFORM 35

Terraform advantages 35

Session-31:20-FEB-2026 39

Git ignore 39

Variables 39

Conditions 39

Precedence of variables 40
loops 40
Session-32: 23-FEB-2026 41
for_each loop 41
Data source 42
Functions 42
Session-33: 24-FEB-2026 43
State and remote state 43
locals 45
Provisioners 45

LINUX

SHELL SCRIPTING

27-JAN-2026 Session-14

Inode Symlink Hardlink

```
[ec2-user@ip-172-31-31-187 ~]$ ls
linux.txt
[ec2-user@ip-172-31-31-187 ~]$ ls -li
total 0
8768976 -rw-r--r--. 1 ec2-user ec2-user 0 Jan 30 11:34 linux.txt
[ec2-user@ip-172-31-31-187 ~]$ stat linux.txt
  File: linux.txt
  Size: 0          Blocks: 0          IO Block: 4096   regular empty file
Device: 10301h/66305d   Inode: 8768976     Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/ec2-user)   Gid: ( 1000/ec2-user)
Context: unconfined_u:object_r:user_home_t:s0
Access: 2026-01-30 11:34:36.344353743 +0000
Modify: 2026-01-30 11:34:36.344353743 +0000
Change: 2026-01-30 11:34:36.344353743 +0000
 Birth: 2026-01-30 11:34:36.344353743 +0000


[ec2-user@ip-172-31-31-187 ~]$ vim linux.txt
[ec2-user@ip-172-31-31-187 ~]$ stat linux.txt
File: linux.txt
  Size: 13         Blocks: 8          IO Block: 4096   regular file
Device: 10301h/66305d   Inode: 8768996     Links: 1
```

Access: (0644/-rw-r--r--) Uid: (1000/ec2-user) Gid: (1000/ec2-user)

Context: unconfined_u:object_r:user_home_t:s0

Access: 2026-01-30 12:58:21.325056651 +0000

Modify: 2026-01-30 12:58:21.325056651 +0000

Change: 2026-01-30 12:58:21.325056651 +0000

Birth: 2026-01-30 12:58:21.325056651 +0000

Inode

Stores metadata of the file/directory

File type -> file/directory/link

Owner group

Permissions

Size

Create, update, etc

Actual memory block address

Inode does not store actual file content.

Symlink == shortlink == softlink

1. Short cut to the file/folder
2. Inode number is different for symlink
3. If actual file is removed, then symlink will break
4. Uses for backward compatibility, switching versions and rollback is easy
5. You can create symlink for folders and files also
6. You can create symlink across different filesystem

Steps:

1. Create one directory(links)
2. Go to link directory
3. Create link for linux.txt
   Command: ln -s
   $ ln -s ../linux.txt link
4. Check link status
   Command: stat link

File: link -> ../linux.txt

Size: 12 Blocks: 0 IO Block: 4096 ***symbolic link***
Device: 10301h/66305d ***Inode: 8768976*** Links: 1

Note: Inode numbers of Symlinks are different

which dnf

/usr/bin/dnf

$ ls -l /usr/bin/dnf

lrwxrwxrwx. 1 root root 5 Feb 7 2025 /usr/bin/dnf -> dnf-3

$ ls -l /usr/bin/yum

lrwxrwxrwx. 1 root root 5 Feb 7 2025 /usr/bin/yum -> dnf-3

YUM -> Before RHEL-8 YUM is the package manager RHEL-8 DNF is the package manager

```
$ touch dnf-3
$ ls
dnf-3  links
$ ln -s dnf-3 dnf
$ ls
dnf  dnf-3  links
$ touch dnf-4
$ ls
dnf  dnf-3  dnf-4  links
$ ls -l
lrwxrwxrwx. 1 ec2-user ec2-user  5 Jan 31 13:20 dnf -> dnf-3
-rw-r--r--. 1 ec2-user ec2-user  0 Jan 31 13:16 dnf-3
-rw-r--r--. 1 ec2-user ec2-user  0 Jan 31 13:25 dnf-4
drwxr-xr-x. 2 ec2-user ec2-user 18 Jan 31 12:13 links

$ ln -sfn dnf-4 dnf
$ ls -l
```

lrwxrwxrwx. 1 ec2-user ec2-user 5 Jan 31 13:26 dnf -> dnf-4
-rw-r--r--. 1 ec2-user ec2-user 0 Jan 31 13:16 dnf-3
-rw-r--r--. 1 ec2-user ec2-user 0 Jan 31 13:25 dnf-4
drwxr-xr-x. 2 ec2-user ec2-user 18 Jan 31 12:13 links

This is the switching versions safely without downtime.

If anything, problem occurs in dnf -4 we can revert back

$ ln -snf dnf-3 dnf
$ ls
dnf dnf-3 dnf-4 links
$ ls -l
total 0
lrwxrwxrwx. 1 ec2-user ec2-user 5 Jan 31 13:46 dnf -> dnf-3
-rw-r--r--. 1 ec2-user ec2-user 0 Jan 31 13:16 dnf-3
-rw-r--r--. 1 ec2-user ec2-user 0 Jan 31 13:25 dnf-4
drwxr-xr-x. 2 ec2-user ec2-user 18 Jan 31 12:13 links

App-

3

App-

4

$ mkdir app-3
ls
app-3 dnf dnf-3 dnf-4 links
$ ln -s app-3 app
$ ls -l
total 0
lrwxrwxrwx. 1 ec2-user ec2-user 5 Jan 31 13:56 app -> app-3
drwxr-xr-x. 2 ec2-user ec2-user 6 Jan 31 13:53 app-3

$ mkdir app-4
$ ln -s app-4 app
$ ls -l
total 0
lrwxrwxrwx. 1 ec2-user ec2-user 5 Jan 31 13:56 ***app -> app-3***

$ ln -sfn app-4 app
$ ls -l

total 0
lrwxrwxrwx. 1 ec2-user ec2-user 5 Jan 31 13:57 ***app -> app-4***

/var/lib/jenkins -> jenkins-

2.8 Jenkins-2.9

/var/lib/jenkins -> jenkins-2.9
Hardlink

1. You can't create hardlink in another filesystem

   $ touch aws.txt

   $ vim aws.txt

   $ ls -l

   -rw-r--r--. 1 ec2-user ec2-user 20 Jan 31 14:18 aws.txt

   $ ln aws.txt /tmp/aws-1.txt

   ln: failed to create hard link '/tmp/aws-1.txt' => 'aws.txt': Invalid cross-device link

2. Hardlink is same as original file but can be different name

   ln aws.txt aws-1.txt

   $ ls -l

   -rw-r--r--. 2 ec2-user ec2-user 20 Jan 31 14:18 aws-1.txt

   -rw-r--r--. 2 ec2-user ec2-user 20 Jan 31 14:18 aws.txt

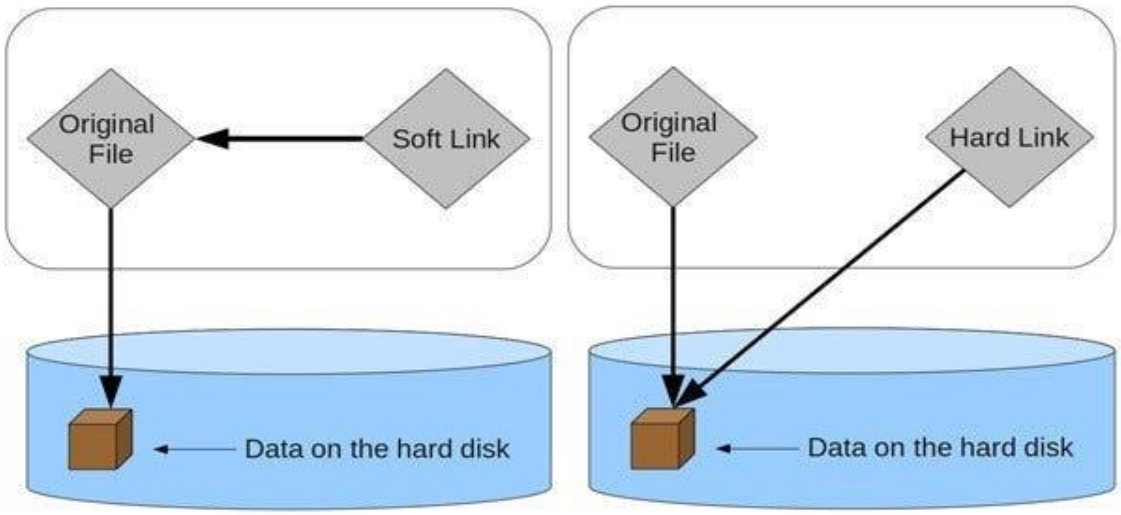
3. Hardlink inode numbers are same.

   $ ls -li

   8769002 -rw-r--r--. 2 ec2-user ec2-user 20 Jan 31 14:18 aws-1.txt

   8769002 -rw-r--r--. 2 ec2-user ec2-user 20 Jan 31 14:18 aws.txt

4. They points to same memory location, so memory is not wasted



5. You can't create hardlink to folders

6. Hardlinks are used for backup.

7. When all links are deleted, then only file will be deleted.

$ rm aws.txt

$ ls -l

-rw-r--r--. 1 ec2-user ec2-user 20 Jan 31 14:18 aws-1.txt

$ cat aws-1.txt

I am learning linux

Using inode number and find command within the same File System.
$ ln aws-1.txt app-3/aws.txt
$ ln aws-1.txt app-4/aws.txt

cd app-3/
$ ls
app-4 aws.txt
$ cd app-4
$ ls
aws.txt
[ec2-user@ip-172-31-28-112 ~]$ ls -li

1639586 drwxr-xr-x. 2 ec2-user ec2-user 34 Jan 31 15:40 app-3
8768999 drwxr-xr-x. 2 ec2-user ec2-user 21 Jan 31 15:40 app-4
8769002 -rw-r--r--. 3 ec2-user ec2-user 20 Jan 31 14:18 aws-1.txt
$ sudo find / -inum *8769002*
/home/ec2-user/app-3/aws.txt
/home/ec2-user/app-4/aws.txt
/home/ec2-user/aws-1.txt

Disadvantages of doing manual

1. Time taking

2. Human errors

3. Everytime from the scratch

4. Can't rely

5. If scale is increased manual way is not possible

Scripting

Commands in a file and run it called scripting.

Where to store scripts?

Version control system

Git -> VCS

1. Version control

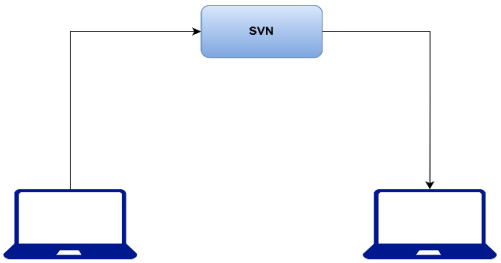
2. Who did the changes, when did the changes

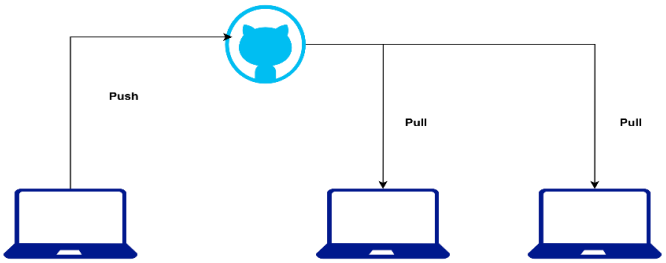
3. You can track who did the changes

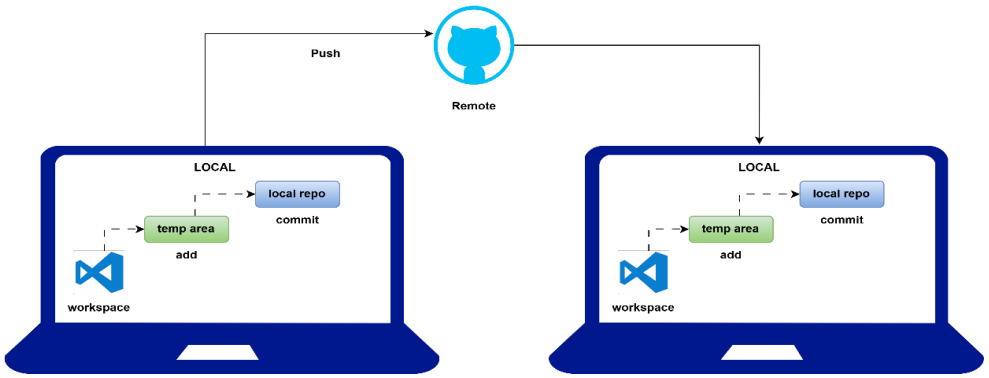
4. Restoring is easy

5. Review is also easy -> Pull Requests

6. Secure place to store the code


SVN


Push
Pull
Pull


Push
Remote
LOCAL
local repo
commit
temp area
add
workspace
LOCAL
local repo
commit
temp area
add
workspace

Centralised vs distributed

SVN -> sub version control

10 files

9 files

Staging/temp area -> add 9 files to staging area

Then commit to local repo

Git commands

1. Clone the repo -> first time download

   Main/master is the default branch

   #!/bin/bash -> shebang

   Shebang is the interpreter of the code in shell script that checks and compiles for errors

   Bash scripting == shell scripting

2. Add files to staging area

   Git add <filename>

   Git config --global user. email "info@joindevops.com" Git config --global user.name "Sivakumar Reddy M"

3. Commit to local repo

   Git commit -m "hello world completed"

   Git push origin main

   Don't edit files inside linux server...

28-JAN-2026 **Session-15**

1. Create repo either in organization or directly

2. Clone repo in laptop
3. write something
4. git add., git add <filename>
5. git commit -m "message"
6. git push origin main

git config --global user.name
git config --global user.email

write only in laptops
don't write in linux servers
only pull linux
95% thinking+design 5% coding
pseudo code -> no syntax, just assumption

```
if today is sunday, enjoy the holiday otherwise go to school
today=Wednesday
if(today != sunday){
        print ("goto school")
}
else{
        print("take holiday, enjoy")
}
```

15
15/2 -> 1
if reminder is 0 -> even
otherwise, odd
number=15

```
reminder=15%2
reminder=1
if(reminder == 0){
        print("even number")
}
else{
        print("odd number")
}
```

variables
data types
conditions
functions
loops
error handling

let x=1, y=0
derive the formula
submit finally
variables are like containers that holds something

DRY -> Don’t repeat yourself
change in one place reflect everywhere you refer
easy to maintain
readable

PERSON1=Suresh
PERSON2=Ramesh
$VAR_NAME

${VAR_NAME}

sh 03-conversation.sh Suresh Ramesh

args == parameters

$1=Suresh

$2=Ramesh

echo "Enter your password"

read -s PASSWORD

read -s encrypts the password

[ec2-user@ip-172-31-28-112 Shell-practice]$ sh 05-Variables.sh

Enter your username::

Sivaram

Username is

Enter your password

I want a command to be executed and take the output into variable, how to do that?

VAR_NAME=$(command)

Special variables

==========

$1 $2 ... $N args passed to script

All variables passed to script: $@

Number of variables passed to script: $#

Script name: $0

Present which directory you are in: $PWD

Who is running this script: $USER

Home direcory of the user: $Home

PID of the script: $$

sleep 100 &

Background process id: $!

Exit status of previous command: $?

$@ vs $*

==========

$@ treats args separately

$* treats as single args

data types

==========

variables are holding data..

mobile number -> numbers

    integers -> -33,768 -> 33,768

    float -> 45.90

    decimanl -> long number

    complex -> 4+8i

names -> string

major? -> yes or no boolean

skills -> devops aws docker kubernetes -> list of skills

skills -> map or dictionary

    devops: 4 -> key -> value

    docker: 3

    kubernetes: 2

areane4 -> 5

speed round up -> 8.2km/sec -> 8km/sec

copied code

integer speed=8km/sec
integer speed=8.2km/sec

number
string

everything is string in shell

position/index in coding starts from 0

conditions
==========

if or when

```
if [ expression ]; then
        code here
fi

if [ expression ]; then
        code here
elif [ expression ]; then
        code here
else
        code here
fi
```

EXIT code

==========

0-127

0 -> success

anything else -> failure

nginx

dnf install nginx -y

if root user then success

otherwise failure

root user id -> 0

other users -> greater than 0

ANSIBLE

Session-21:05-FEB-2026

Create node-1 server and we can login to that server from shell server.

While login into node from shell server we can create files

ssh ec2-user@172.31.24.169 'touch /tmp/shell.txt'

go to shell server check whether the file is create or not.

[ec2-user@ip-172-31-24-169 /tmp]$ ls -l

-rw-r--r-- 1 ec2-user ec2-user 0 Feb 5 19:27 shell.txt

This process is called remote login

I want to run mango DB in node1 server

Without login mangodb server we can do from the node server

How can we copy from one server to another server?

Using ***scp*** (source copy) command we can copy from one server to another server.

1. Copy mongo repo, mongodb.sh

   [ec2-user@ip-172-31-28-112 shell-roboshop]$ scp mongo.repo mongodb.sh ec2-user@172.31.24.169:/tmp

   Go to node server check whether those files are copied or not.

   [ec2-user@ip-172-31-24-169 /tmp]$ ls

   mongo.repo

   mongodb.sh

2. Execute them from server 1

   [ec2-user@ip-172-31-28-112 shell-roboshop]$ ssh ec2-user@172.31.24.169 'cd /tmp && sudo sh mongodb.sh'

Eg:

If we have 100 servers, we can put them into a for loop in a script and execute it, but this process increases complexity and takes a lot of time. To address this issue, a configuration management tool such as **Ansible** can be used.

Configuration management

provisioning server (plane server) -> baseline it according to our needs

1. Installing programming language
2. Creating system users and folders
3. Installing some system packages
4. Downloading code
5. Installing dependencies
6. Creating systemctl services
7. Enable and starting the services

Getting ready the server to host the application is called configuration.

How to be managing the configuration?

CRUD – Creating, Reading, Updating, and Deleting the configuration.

There are 2 types of configuration management tools

1. Push
2. Pull

Push vs Pull

HYD -> Delhi

1. Person in Delhi go to DTDC office every day and check for letter.
2. Person stays home, DTDC delivers letter whenever it arrives.

Ansible diagram

Install ansible in Ansible server

sudo dnf install ansible -y

To establish a connection with the server, the following details are required: protocol, port, username, password, and IP address.

Step 1 :Using Ansible connection establish with server

```
[ec2-user@ip-172-31-28-112 ~]$ ansible all -i 172.31.24.169, -e ansible_user=ec2-user -e ansible_password=DevOps321 -m ping
172.31.24.169 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
```

```
[ec2-user@ip-172-31-28-112 ~]$ ansible all -i 172.31.24.169, -e ansible_user=ec2-user -e ansible_password=DevOps321 -b -m dnf -a"name=nginx state=installed"
172.31.24.169 | CHANGED => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": true,
    "msg": "",
    "rc": 0, (return code or exit code)
    "results": [
        "Installed: redhat-logos-httpd-90.5-1.el9_6.1.noarch",
```

```
        "Installed: nginx-2:1.20.1-22.el9_6.3.x86_64",
        "Installed: nginx-core-2:1.20.1-22.el9_6.3.x86_64",
        "Installed: nginx-filesystem-2:1.20.1-22.el9_6.3.noarch"
    ]
}
```

Linux commands == Ansible modules/collections

Installing nginx in node1 server from Ansible server using ansible commands

Linux Server STATE-1 -> STATE change STATE-2

```
[ec2-user@ip-172-31-28-112 ~]$ ansible all -i 172.31.24.169, -e ansible_user=ec2-user -e ansible_password=DevOps321 -b -m dnf -a"name=nginx state=installed"
172.31.24.169 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "msg": "Nothing to do",
    "rc": 0,
    "results": []
}
```

Start the nginx

```
[ec2-user@ip-172-31-28-112 ~]$ ansible all -i 172.31.24.169, -e ansible_user=ec2-user -e ansible_password=DevOps321 -b -m service -a"name=nginx state=started"
```

YAML -> Yet another markup language

DTO -> data transfer objects

name -> sivakumar

acc no

amount

date

branch

denomination

forms and templates

Markup languages

HTML, XML

```
<Register>
        <Name>Sivakumar</Name>
        <Mobile>12345678</Mobile>
        <Email>info@joindevops.com</Email>
</Register>
```

JSON

YAML

inventory

what are the servers ansible can connect to

ansible is in python

we give inputs in playbook, ansible runs inside python connect to the systems using API

Clone the Ansible repo into Ansible server

[ec2-user@ip-172-31-29-20 ~/Ansible]$ ansible ***frontend*** -i inventory.ini --list-hosts

hosts (3):

3.80.68.41

3.80.68.42

3.80.68.43

[ec2-user@ip-172-31-29-20 ~/Ansible]$ ansible ***all*** -i inventory.ini --list-hosts

hosts (4):

3.80.68.44

3.80.68.41

3.80.68.42

3.80.68.43

[ec2-user@ip-172-31-29-20 ~/Ansible]$ ansible ***roboshop*** -i inventory.ini --list-hosts

hosts (3):

3.80.68.41

3.80.68.42

3.80.68.43

[ec2-user@ip-172-31-29-20 ~/Ansible]$ ansible ***ungrouped*** -i inventory.ini --list-hosts

hosts (1):

3.80.68.44

[ec2-user@ip-172-31-29-20 ~/Ansible]$ ansible-inventory -i inventory.ini --graph

@all:

|--@ungrouped:

| |--3.80.68.44

|--@roboshop:

| |--@frontend:

| | |--3.80.68.41

| | |--3.80.68.42

| | |--3.80.68.43

| |--@backend:

| | |--3.80.68.41

| | |--3.80.68.42

| | |--3.80.68.43

Ansible is used to configure and manage servers, and it can also provision (create) servers using cloud and infrastructure modules.

Now we are connecting the node1 server from ansible server

Step 1: Provide the node1server private ip in inventory

```
[web]
172.31.27.213
```

And update the playbook.yml

```
- name: ping the server
  hosts: web
  tasks:
  - name: ping the web server
    ansible.builtin.ping:
```

and run the command in ansible server

[ec2-user@ip-172-31-29-20 ~/Ansible]$ ansible-playbook -i inventory.ini -e ansible_user=ec2-user -e ansible_password=DevOps321 01-Playbook.yml

```
100.27.204.175 | 172.31.29.20 | t3.micro | https://github.com/DevSecOps-ssr/Ansible.git
[ ec2-user@ip-172-31-29-20 ~/Ansible ]$ ansible-playbook -i inventory.ini -e ansible_user=ec2-user -e ansible_password=DevOps
321 01-Playbook.yml

PLAY [ping the server] *********************************************************************************************

TASK [Gathering Facts] *********************************************************************************************
ok: [172.31.27.213]

TASK [ping the web server] *****************************************************************************************
ok: [172.31.27.213]

PLAY RECAP *********************************************************************************************************
172.31.27.213              : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

[ec2-user@ip-172-31-29-20 ~/Ansible]$ ansible-playbook -i inventory.ini -e ansible_user=ec2-user -e ansible_password=DevOps321 02-Playbook.yml

```
100.27.204.175 | 172.31.29.20 | t3.micro | https://github.com/DevSecOps-ssr/Ansible.git
[ ec2-user@ip-172-31-29-20 ~/Ansible ]$ ansible-playbook -i inventory.ini -e ansible_user=ec2-user -e ansible_password=DevOps
321 02-Playbook.yml

PLAY [Hello World] *************************************************************************************************

TASK [Gathering Facts] *********************************************************************************************
ok: [172.31.27.213]

TASK [print Hello World] *******************************************************************************************
ok: [172.31.27.213] => {
    "msg": "Hello world!"
}

PLAY RECAP *********************************************************************************************************
172.31.27.213              : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

In Red Hat–based Linux systems, packages are installed using ***dnf***, and in Ubuntu-based systems, packages are installed using ***apt***. The ***ansible.builtin.package*** module automatically uses dnf for Red Hat systems and apt for Ubuntu systems.

Step2: Using Install and run nginx.

```
100.27.204.175 | 172.31.29.20 | t3.micro | https://github.com/DevSecOps-ssr/Ansible.git
[ ec2-user@ip-172-31-29-20 ~/Ansible ]$ ansible-playbook -i inventory.ini -e ansible_user=ec2-user -e ansible_password=DevOps
321 03-Playbook.yml

PLAY [Install and run nginx] ***************************************************************************************

TASK [Gathering Facts] *********************************************************************************************
ok: [172.31.27.213]

TASK [Install nginx] ***********************************************************************************************
ok: [172.31.27.213]

PLAY RECAP *********************************************************************************************************
172.31.27.213              : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Note: If we give local connection, it does not expect user id and password.

Session-23:09-FEB-2026

Variables from argos

09-Vars.yml (Arguments)

[ec2-user@ip-172-31-79-249 ~/Ansible $ ansible-playbook -i inventory.ini -e Name=Shiva 09-Vars.yml

```
[ ec2-user@ip-172-31-79-249 ~/Ansible ]$ ansible-playbook -i inventory.ini -e Name=Shiva 09-Vars.yml

PLAY [variables from args] *****************************************************************************

TASK [Gathering Facts] *********************************************************************************
ok: [localhost]

TASK [print variables from args] ***********************************************************************
ok: [localhost] => {
    "msg": "Hi Shiva, Good morning!"
}

PLAY RECAP *********************************************************************************************
localhost                  : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

```
100.27.204.175 | 172.31.29.20 | t3.micro | https://github.com/DevSecOps-ssr/Ansible.git
[ ec2-user@ip-172-31-29-20 ~/Ansible ]$ ansible-playbook -i inventory.ini -e ansible_user=ec2-user -e ansible_password=DevOps
321 03-Playbook.yml

PLAY [Install and run nginx] ***************************************************************************

TASK [Gathering Facts] *********************************************************************************
ok: [172.31.27.213]

TASK [Install nginx] ***********************************************************************************
ok: [172.31.27.213]

TASK [start nginx] *************************************************************************************
ok: [172.31.27.213]

PLAY RECAP *********************************************************************************************
172.31.27.213              : ok=3    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Variables preference

1. Args

   [ec2-user@ip-172-31-24-97 ~/Ansible]$ ansible-playbook -i inventory.ini -e ***'Name="Ansible from args"'*** 10-vars\ preference.yml

   TASK [print name]

   ok: [localhost] => {

   "msg": "Hi Ansible from args"

   }

2. Tasks

   [ec2-user@ip-172-31-24-97 ~/Ansible]$ ansible-playbook -i inventory.ini 10-vars\ preference.yml

   Enter your Name: Sivaram

   TASK [print name]

   ok: [localhost] => {

   "msg": "Hi Ansible from task"

}

3. File

   [ec2-user@ip-172-31-24-97 ~/Ansible]$ ansible-playbook -i inventory.ini 10-vars\ preference.yml

   Enter your Name: shiva

   ok: [localhost] => {

   "msg": "Hi Ansible from file"

   }

4. Prompt
5. Play
6. Inventory

   ok: [localhost] => {
   "msg": "Hi Ansible from inventory"
   }

Finally, it doesn't find any variables then it gives undefined variable

Data types

```
PROGRAM: "DevSecOps with AWS"        # string
  DURATION: 140                      # int/number
  IS_LIVE: true                      # bool
  DETAILS:                           # map/dictionary {}
    MONTHS: 4
    TRAINER: "Sivakumar Reddy"
  TOPICS:                            # list -/[]
  - Linux
  - Shell
  - Ansible
```

Conditions

In Ansible, conditions are used to determine whether a task should run.

vars:

```
    IS_LIVE: true

ok: [localhost] => {
    "msg": "Yes, it is Live"
}

vars:
    IS_LIVE: false

ok: [localhost]

no output
```

- **ansible_facts** is a reserved variable name in Ansible.
- It is a special, top-level variable used by Ansible to store the gathered system information (facts) about managed nodes.
- **Purpose:** It holds the dictionary of facts (OS, IP address, hardware, etc.) collected by the setup module or other modules.

Shell vs command module

ansible.builtin.command vs ansible.builtin.shell

command
simple commands you can execute
you can't get complete shell env of the remote server. more secure

>, |, will not work here

shell
you will get the entire environment of the server...
less secure
> | works here

ssh ec2-user@IP 'echo "Hello" > /tmp/hello.txt'

When you don't find any module available for your requirement, we can use command module or shell module

Loops
Item is a reserve keyword.

```
#List of string
loop:
  - mysql
  - nginx
  - zip

#List of map
loop: # list of map
  - { name: "mysql", state: "absent"} # map/dict
  - { name: "nginx", state: "present"}
  - { name: "zip", state: "present"}
```

Register output

In Ansible, the register keyword is used to capture the output of a task into a variable. This allows you to inspect the results, check for success or failure, and use specific return values in subsequent tasks within the same play.

```
- name: (Task description)
  (module name): (module arguments)
  register: (variable_name)
```

Functions
We will do specific task by taking inputs.
There are no functions in ansible instead of function we have filters.

Filters

Syntax: VAR_NAME | FILTER_NAME

convert yaml to json

1. take yaml into one variable
2. then convert to json

```
- name: read students yaml
   ansible.builtin.set_fact: # setting the variable
     student_yaml: "{{ lookup('file', 'students.yaml' ) | from_yaml }}"
```

Session-24:10-FEB-2026

1. play level
2. task level
3. files
4. prompt
5. inventory
6. command line

register
set_fact

shell vs command

conditions -> when
loops -> loop -> item
list -> -/[
dict -> {

filters/functions
VAR | filter

pip is the package installer for Python. You can use pip to install packages from the Python Package Index and other indexes.

ansible –version
ansible python module location = /usr/lib/python3.9/site-packages/ansible

we have to install 3.9 version package
pip3.9 install netaddr

netaddr refers to a third-party Python library that provides a set of tools for manipulating and evaluating network data, such as IP addresses, subnets, and MAC addresses.
netaddr is a Python package that must be installed separately on the Ansible **controller node** (the machine where you run ansible-playbook)

Error handling

ignore_errors: true

Idempotency

Ansible ensures ***idempotency*** when you use proper modules (including those from collections), but the command and shell modules are not idempotent by default.

In Ansible, idempotency is achieved when we use proper modules (including modules from collections).
Rule: If a suitable collection module is available, we should use it to ensure idempotency. If no such module is available, we may use the command or shell module, but we must handle idempotency manually.

Playbook is list of place, which can execute the task against remote server.
Using ansible we can install any packages

EC2 and R53 creation using ansible
Roboshop using ansible

Without inventory file
hosts: all
connection: local
ansible-playbook -i localhost, roboshop.yml
catalogue-dev.daws88s.online
catalogue-qa.daws88s.online

1. create instances
2. create r53 records

1. console
2. cli

3. SDK -> botocore and boto3 (pip 3.9 install botocore boto3)

Session-29:18-FEB-2026

Include vs import role

import_role and include_role

1. tags applied to import_role will be automatically applied to all the tasks inside it.
2. tags applied to include_role will not be cascaded.
3. import_roles parse the playbook before execution, it is static. include_role is dynamic, it will directly execute the playbook, it will not parse like import_role before execution.
4. conditions applied to import_role is applied to all the tasks, include_role will not care the condition applied to it.
5. You cannot use loops on 'import_role' statements. You should use 'include_role' instead

Dynamic inventory

scaling environment, servers are dynamically created and destroyed based on traffic.. we cant use static inventory file, we need to dynamically query the instances. We can use dynamic inventory plugin from AWS passing region, name of the instances, runnning state, etc.

go to us-east-1, search frontend-dev running, get the IP address

Vault

ansible-vault create valut.yaml
code -> configuration(confidential, non-confidential)
AWS -> Platform

Parallel execution

forks = 8
each task is targeted on 8 servers
single playbook execution

serial=5
8 forks -> 5
run entire playbook
next 3 servers

Ansible disadvantages

Ansible does not have the state
Ansible is not useful for the infrastructure creation because it doesn't have state.
We can create infrastructure, but it doesn't manage the infrastructure.
IaaC

I created infra -> I keep track of it in store
again, I asked for some change -> compare state and proceed.

Ansible is a perfect configuration management tool. Within the Linux server, Ansible can do anything, but if you use Ansible for the infrastructure creation, since it has not, you don't have any state file, it will be failed to manage the infra. But Terraform has a state file. It will be used to manage the infra.

Terraform -> creating servers
Ansible -> connect to servers and configure them

If we properly integrate the terraform and ansible and then we can get the fully automation.

Session-30:19-FEB-2026

Ansible -> node
node -> public key
Ansible -> private key
ssh -i private-key ec2-user@IP

servers -> create one user for ansible

get the private key access to ansible server
configure the path in ansible.cfg
make sure keys are RSA based

TERRAFORM

IaaC tool
Terraform alternatives: Pulumi, Cloudformation (AWS), Arm, etc...

multi-cloud
Terraform can connects to any platform

Why should we use Iaac tool? And what are the disadvantages to create infra manually?

Terraform advantages

1. Version control

   We can have history of our infra...easy to restore. can track who changed and review

   Eg: When we configure applications manually, we don't have version control, so we can't track our infrastructure. For example, if someone login to the server by mistake and modifies something, it is not easy to troubleshoot (can't track who changed and review).

2. Consistent infra

   DEV, UAT and PROD

Eg: Generally, problems occur when the application works fine in the development environment but does not work in production. Therefore, we cannot create consistent infrastructure manually.

3. CRUD -> inventory management

   By using Terraform, we can track and manage all the infrastructure resources defined in our code. not only create infra but we can read, update and delete the infra.

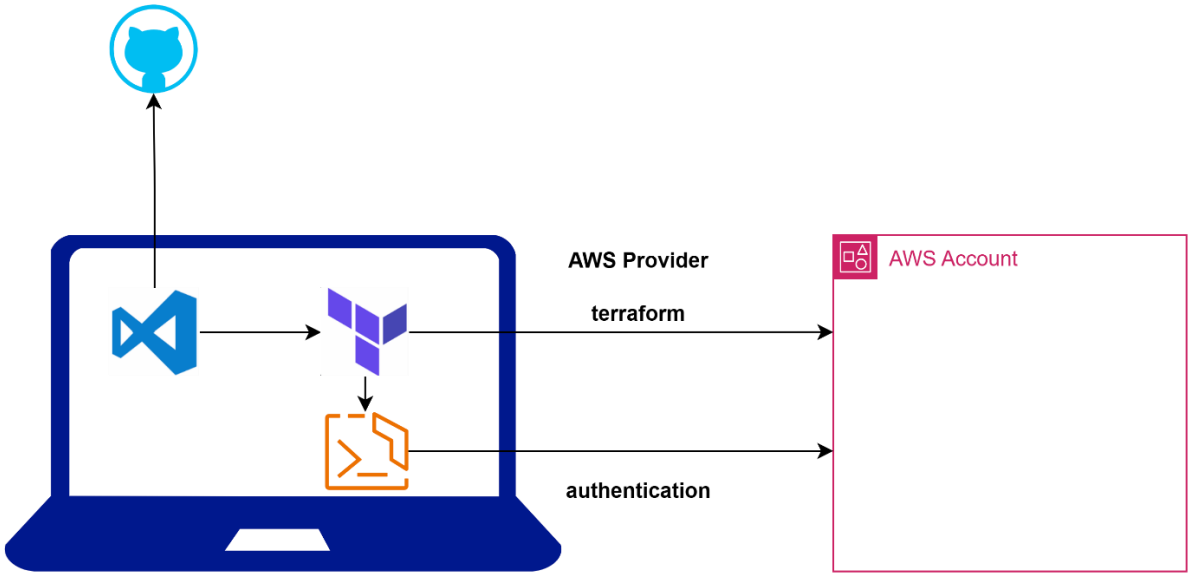
4. Cost optimization

   We can delete dev servers after business servers, and we can create servers next day

5. Dependency management (security groups …)-> automatic in terraform

6. Reusable infra -> terraform modules

1. Install terraform
2. Setup the PATH in ENV variables
3. Install AWS CLI V2
4. Run aws configure.



Providers

Terraform connect to not only cloud providers like Aws, Gcp and Azure it can connect to git hub, On-premise. For reference go through below link.
https://registry.terraform.io/browse/providers

A provider.tf file in Terraform is used to **declare and configure the specific provider plugins** that Terraform needs to interact with various platforms and services, such as cloud providers (AWS, Azure, Google Cloud), SaaS products, or other API

HCL -> Hashicorp configuration language

```
block_name "type" "name" {

}
```

resource, data, output, variable, module, terraform, etc..

```
resource "aws_instance" "name-of-the-instance" {
        key = value
}
```

When you execute the above script, it calls a function in go language for that function we are passing some inputs.

aws_instance -> syntax
name-of-the-instance -> we can change, this is for terraform reference
ami, name, sg, instance_type

Terraform is developed by using Go Language

Creating Ec2 instance

Steps:

1. First, we should create provider.tf

2. And we have to create ec2.file (mention terraform) code which resource you want).
3. And apply the below commands.

   Terraform init

   Terraform plan

   Terraform apply.
4. Finally, resource will be created in Aws.

terraform init => initialize terraform

During terraform init, Terraform checks the terraform block for required providers. If the AWS provider is defined, Terraform downloads the latest compatible version of the AWS provider plugin based on the version constraints.

terraform plan

terraform plan is used to show what resources Terraform will create, modify, or delete before applying the changes.

Or

The terraform plan command generates an execution plan that displays the changes Terraform will make to the infrastructure, including resources to be created, updated, or destroyed.

terraform apply

terraform apply is used to execute the changes defined in the Terraform configuration and create, update, or delete infrastructure resources.

terraform apply -auto-approve

Session-31:20-FEB-2026

- Conditions
- Count based loop

terraform init -> downloads the provider
terraform plan -> plan the infra but it will not create
terraform apply -> create the infra
terraform destroy -> destroy the infra

Git ignore

Note:
In git we have to store only code not .exe software's. it does not allows the .exe files.

```
resource "type_of_resource" "name_of_resource"{
        key = value # arguments
}
```

attributes -> outputs

```
rm -rf .git
git init
git branch -M main
git remote add origin <URL>
git add . ; git commit -m ""; git push origin main
```

Variables

```
variable "name_of_variable" {
        default = ""
}
```

Conditions

```
expression ? "true-value" : "false-value"
```

1. default values inside variables.tf
2. terraform.tfvars

T3.small

3. command line
4. env variables

   Set TF_VAR_sg_name = “allow-terraform-env”

Precedence of variables

1. command line
   terraform plan -var="sg_name=allow-all-terraform-cmd"
2. tfvars
3. env variables -> export TF_VAR_VAR_NAME="value"
   export TF_VAR_sg_name="allow-all-terraform-env"
   unset TF_VAR_sg_name
4. default values

loops

1. count based loops
2. for_each
3. dynamic block

1. sg and ec2 instances -> 10 instances
2. r53 records

count.index

```
output "name_of_the_output" {
        value =
}
```

cp Variables/ec2.tf Variables/provider.tf Variables/variables.tf Conditions/

If operate system gets any update automatically will be updated so we should not do hardcore ami id in terraform code.

Session-32: 23-FEB-2026

```
resource "type_of_resource" "name_of_resource"{
        key=value
}

variable "name_of_variable" {
        type = string
        default = ""
}
```

terraform.tfvars
ENV variables TF_VAR_VAR_NAME
-var "var_name=value"

expression ? "true_val" : "false_val"

count.index

list -> ordered, index based values -> allows duplicates
set -> order not guarantied, remove duplicates

count based loop -> list

for_each loop

set or map
each -> special variable

count, for_each, dynamic (only for repeated code inside resource)
count -> list based
for_each -> map or set

Terraform dynamic blocks are used to generate multiple, repeatable nested blocks within a resource, data source, provider, or provisioner block. This feature helps you avoid code repetition, follow the DRY (Don't Repeat Yourself) principle, and create more scalable and maintainable configurations, especially in cloud environments like AWS where similar configurations (e.g., security group rules) are common.

Data source

Data source in Terraform allows you to **dynamically fetch the ID of an Amazon Machine Image (AMI)** based on specific criteria, such as name patterns and owners. This prevents the need to hardcode AMI IDs, making your configurations more portable across different regions and ensuring you can always use the latest image.

If you want to get the information from provider, we can use data sources

Functions

we can't write custom functions in terraform.

FName
MName
LName

merge

```
{
        a = "b"
        c = "d"
}

{
        d = "e",
        c = "z"
}

{
        d = "e",
        e = "f"
}

a = "b"
```

c = "d"
d = "e"
e = "f"

Finding value of a key in a map terraform
Finding element index in list

Session-33: 24-FEB-2026

count based loop
- list
- count.index

for based loop
- set or map
- each, each.key and each.value

dynamic block
- we get special variable with block name, for example ingress

data sources
- query the provider for existing information

functions

State and remote state

terraform stores/tracks what it had created in the state file, it is like memory to terraform

.tf files -> desired/declared/expected/asked infra
actual infra -> what is actually creted inside provider
state -> what terraform created

desired infra == actual infra

first time

declare infra
actual infra not created

state is empty

terraform apply
terraform creates the resources in provider and note them in state file
apply again
.tf files, state file, actual infra
state file == actual infra

scan .tf files

.tf files == actual infra

update tf files
.tf, actual infra, state file
state file == actual infra -> means nothing changed outside of terraform

.tf files!= actual infra

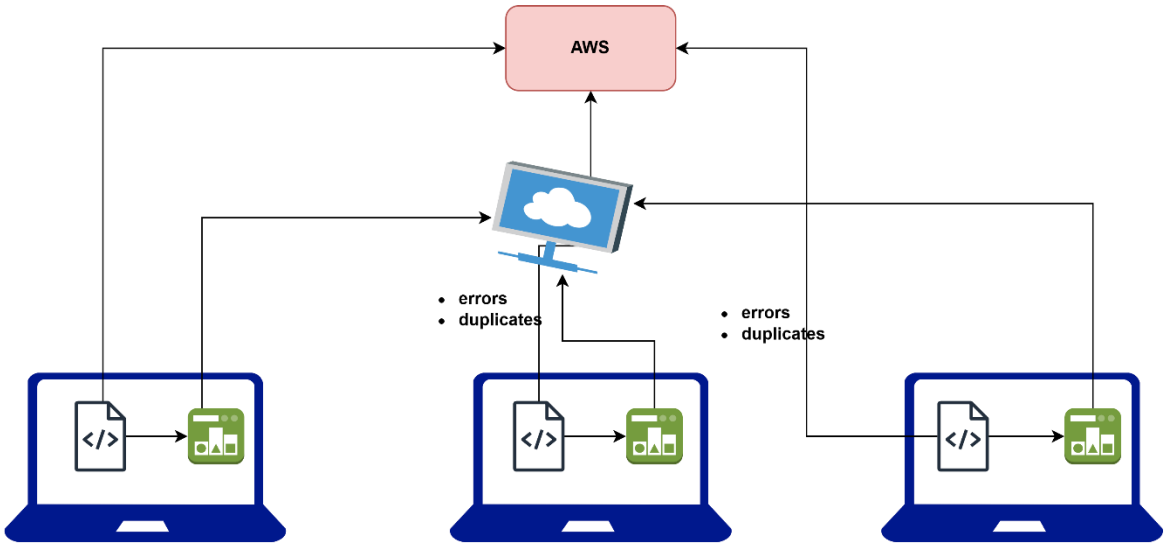
changes outside terraform
.tf, actual infra, state file
state file! = actual infra -> means someone changed outside of terraform
desired! = actual infra

destroy
delete the infra, update state file
state file -> for terraform
tf files -> us
state file is locked by terraform when it is performing actions on that

remote state
in collaboration environment, state should be stored in remote like S3 so that we can prevent errors and duplicates, it should be locked parellel modifications should not be allowed

remote state secure

1. users should not have access to delete except terraform
2. you should enable versions
3. data replication in another bucket



locals

locals are like variables but has extra capabilities
1. you can refer other variables inside locals
2. values inside locals cant be overridden.
3. we can store functions or expressions in locals

Provisioners

1. local-exec -> where terraform executes
2. remote-exec -> executes inside the resources created by terraform

if we want to take some actions or run scripts then we can use provisioners

provisioners will be executed at the time of destroy or creation but not at the time of updating the resources

openssl ssh version mismatch